

# Sistema automatizado de controle de iluminação e temperatura baseado em presença

Guia do protótipo: o que foi construído, como funciona por dentro e como usar o aplicativo

---

**Autores:** Alex Gabriel Alves Machado e Gabriel Pio da Silva

**Orientadora:** Profa. Dra. Vanessa Santos Lessa

Trabalho de Curso II

# Sumário

---

- 1** O trabalho em resumo  
Tema, método e o que o protótipo entrega
- 2** Arquitetura do sistema  
Do sensor ao dashboard: as peças e como conversam
- 3** A lógica de ocupação  
A máquina de estados da Figura 3 e as regras de luz e HVAC
- 4** Instalação  
macOS, Windows e a partir do código-fonte
- 5** Usando o aplicativo  
Controladora, ajustes, modo Demo, Histórico e Configurações
- 6** Firmware real no Wokwi  
O ESP32 simulado enviando dados para o aplicativo
- 7** Métricas e exportação  
O que é medido para o Capítulo 4 e o formato do CSV
- 8** Contrato MQTT  
Tópicos e mensagens trocados entre dispositivo e aplicativo
- 9** Decisões técnicas  
O que mudou em relação ao modelo do TC I, e por quê
- 10** Limitações  
O que o protótipo não cobre e deve ser declarado no texto
- 11** Para quem vai mexer no código  
Estrutura do repositório e comandos

# 1 O trabalho em resumo

O projeto automatiza a iluminação e a climatização de uma sala a partir da presença de pessoas: a luz só acende e o ar-condicionado só liga quando há alguém no ambiente, e desligam sozinhos quando a sala fica vazia.

## Método

O trabalho combina **revisão integrativa de literatura** com uma **prova de conceito**. O TC I propôs um modelo arquitetural preliminar (Figuras 2 e 3 do pré-projeto). No TC II esse modelo é tratado como hipótese:

1. o protótipo implementa o modelo do TC I tal como foi proposto;
2. a revisão integrativa (literatura de 2016 a 2026) confronta esse modelo;
3. os ajustes no protótipo que a revisão justificar são o resultado do trabalho;
4. o Capítulo 4 compara o protótipo inicial, o ajustado e os valores da literatura.

## O que o protótipo entrega

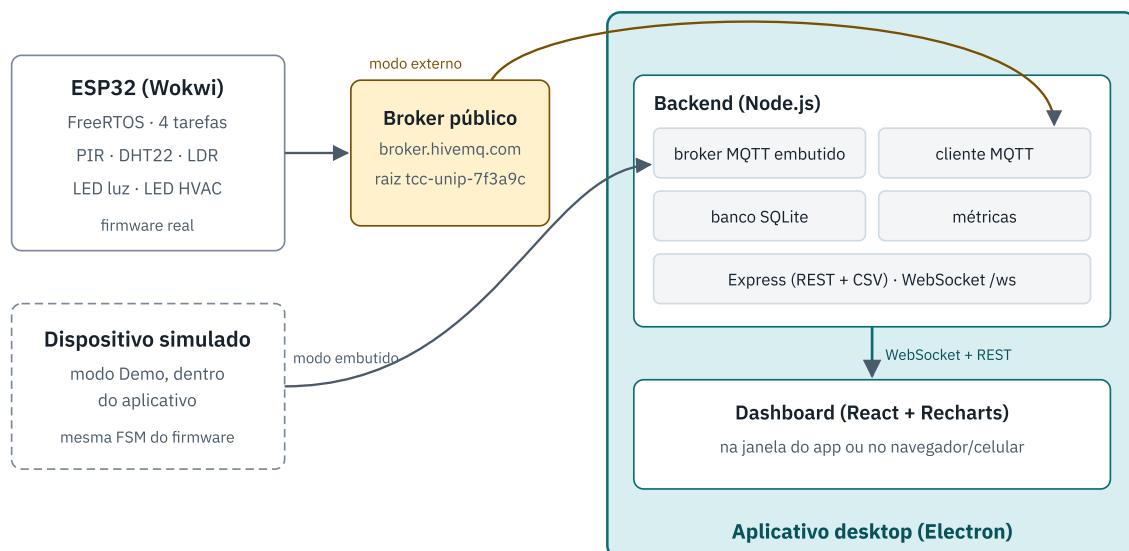
| Parte                      | O que faz                                                                                                                                                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Firmware do ESP32</b>   | Lê o sensor de presença (PIR), temperatura e umidade (DHT22) e luminosidade (LDR); decide pela máquina de estados; aciona a luz e o ar-condicionado; publica o estado por MQTT. Roda no simulador Wokwi. |
| <b>Aplicativo desktop</b>  | Instalável no macOS e no Windows. Recebe as leituras, guarda o histórico, calcula as métricas e mostra tudo num painel com o diagrama do circuito e da máquina de estados.                               |
| <b>Modo Demo</b>           | Um dispositivo simulado dentro do aplicativo, com a mesma máquina de estados do firmware, para demonstrar o sistema sem Wokwi, sem internet e sem hardware.                                              |
| <b>Exportação de dados</b> | Todas as leituras exportáveis em CSV, com latência e perda de mensagens por leitura, para as análises do Capítulo 4.                                                                                     |

### Por que simulação e não hardware?

O trabalho não usa placa nem sensores físicos: o ESP32 roda no Wokwi, um simulador de microcontroladores. O código do firmware é o mesmo que rodaria numa placa real; o que muda é que não há ruído de sensor nem a rede local de um prédio. Essa escolha é uma limitação declarada (seção 10).

## 2 Arquitetura do sistema

A arquitetura é a do TC I: **dispositivo** → **MQTT** → **backend Node/Express** → **WebSocket** → **dashboard React**. O aplicativo desktop apenas empacota o backend e o dashboard num programa só, para abrir com dois cliques.



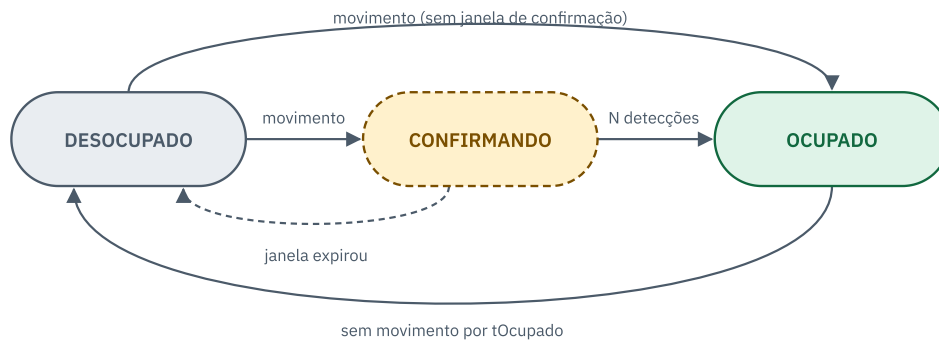
**Figura 1.** Arquitetura do protótipo. O firmware real publica no broker público; o dispositivo simulado (Demo) publica no broker embutido. Em ambos os casos o backend recebe como um cliente MQTT comum.

### As peças

| Peça                                  | Papel                                                                                                                                                                                                                                                                                                      |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Firmware</b><br>C++ · FreeRTOS     | Quatro tarefas: <i>aquisição</i> (lê os sensores), <i>decisão</i> (roda a máquina de estados a cada 100 ms), <i>atuação</i> (acende e apaga os LEDs) e <i>comunicação</i> (Wi-Fi, MQTT e horário via NTP). Publica o estado a cada 2 s e a cada mudança, e aceita novos parâmetros por MQTT sem reiniciar. |
| <b>Broker MQTT</b>                    | Intermedia as mensagens. O aplicativo tem um broker próprio (embutido, não precisa instalar nada) e também sabe se conectar a um broker externo, como o público usado pelo firmware no Wokwi.                                                                                                              |
| <b>Backend</b><br>Node.js · Express   | Recebe e valida as mensagens, calcula latência e perda, grava no banco SQLite, serve a API REST e a exportação CSV, e transmite as novidades ao dashboard em tempo real por WebSocket.                                                                                                                     |
| <b>Dashboard</b><br>React · Recharts  | Três telas: Controladora (tempo real), Histórico (gráficos e métricas) e Configurações. O mesmo endereço abre no navegador de outro computador ou celular da rede local.                                                                                                                                   |
| <b>Aplicativo desktop</b><br>Electron | Liga o backend quando o programa abre, mostra o dashboard numa janela e desliga tudo de forma limpa ao fechar. É o que se instala a partir do <code>.dmg</code> ou do <code>.exe</code> .                                                                                                                  |

### 3 A lógica de ocupação

O coração do sistema é a máquina de estados da Figura 3 do TC I, que decide se a sala está ocupada. Ela é a mesma no firmware (C++) e no modo Demo (TypeScript), e os dois passam pelo mesmo conjunto de 9 testes.



**Figura 2.** Máquina de estados de ocupação. O estado CONFIRMANDO (tracejado) só existe quando a janela de confirmação está ativa; com ela desligada (padrão), o modelo é exatamente o da Figura 3 do TC I.

#### Como os estados mudam

- **DESOCUPADO** → ao detectar movimento, a sala passa direto a **OCUPADO** (modelo do TC I).
- Com a **janela de confirmação** ligada, o primeiro movimento leva a **CONFIRMANDO**: a ocupação só é aceita se houver N detecções dentro da janela; se a janela acabar antes, a detecção é descartada e a sala volta a **DESOCUPADO**. Isso reduz falsos positivos (por exemplo, alguém que só passou pela porta).
- Em **OCUPADO**, cada movimento renova o prazo. Sem movimento por **t0ocupado**, a sala volta a **DESOCUPADO**.

#### Regras dos atuadores

**Luz (LED amarelo, representa um relé)**

Acende quando a sala está **ocupada** e a luminosidade está **abaixo do limiar**.

**HVAC (LED azul, representa o comando IR)**

Liga quando a sala está **ocupada** e a temperatura está **acima da temperatura-alvo**.

## Parâmetros ajustáveis

Todos podem ser alterados com o sistema rodando, pelo aplicativo (seção 5.2) ou por MQTT. Valores de fábrica:

| Parâmetro    | Padrão           | Significado                                                               |
|--------------|------------------|---------------------------------------------------------------------------|
| tOcupadoMs   | 30 s             | Tempo sem movimento até a sala voltar a DESOCUPADO.                       |
| janelaConfMs | 0<br>(desligada) | Duração da janela de confirmação. Com 0, não existe o estado CONFIRMANDO. |
| pulsosConf   | 1                | Quantas detecções dentro da janela confirmam a ocupação.                  |
| luxLimiar    | 300 lux          | Abaixo deste valor a luz pode acender.                                    |
| tempAlvo     | 26 °C            | Acima deste valor o HVAC pode ligar.                                      |

## 4 Instalação

Os instaladores ficam na página de **Releases** do repositório no GitHub ( [github.com/mvacoimbra/tcc-automacao/releases](https://github.com/mvacoimbra/tcc-automacao/releases) ). O aplicativo não precisa de internet, Docker nem de nenhum outro programa para funcionar no modo Demo.

### macOS (Apple Silicon)

- 1 Baixe `tcc-automacao-0.1.0-macos-arm64.dmg` e abra o arquivo.
- 2 Arraste **TCC Automacao** para a pasta **Aplicativos**.
- 3 Na primeira vez, abra pelo menu de contexto: clique com o botão direito no app → **Abrir** → confirme.
- 4 Se o macOS disser que o app “está danificado” ou não pode ser aberto, rode no Terminal:  

```
xattr -dr com.apple.quarantine "/Applications/TCC Automacao.app"
```

### Windows (64 bits)

- 1 Baixe `tcc-automacao-0.1.0-windows-x64-setup.exe` e execute.
- 2 No aviso do SmartScreen, clique em **Mais informações** → **Executar assim mesmo**.
- 3 Siga o instalador; o aplicativo fica no menu Iniciar como **TCC Automacao**.

#### Por que os avisos de segurança?

Os instaladores não têm assinatura digital de desenvolvedor (um certificado pago, fora do escopo do trabalho), então o sistema operacional avisa antes de abrir. O programa é o mesmo que está no repositório; o aviso aparece só na primeira execução.

### Onde ficam os dados

O histórico e as configurações ficam numa pasta do usuário e continuam lá entre uma abertura e outra:

- macOS: `~/Library/Application Support/@tcc/desktop/`
- Windows: `%APPDATA%\@tcc\desktop\`

Apagar essa pasta zera o histórico e volta às configurações de fábrica.

### A partir do código-fonte

Para desenvolvimento, com Node 24 e pnpm 11 instalados:

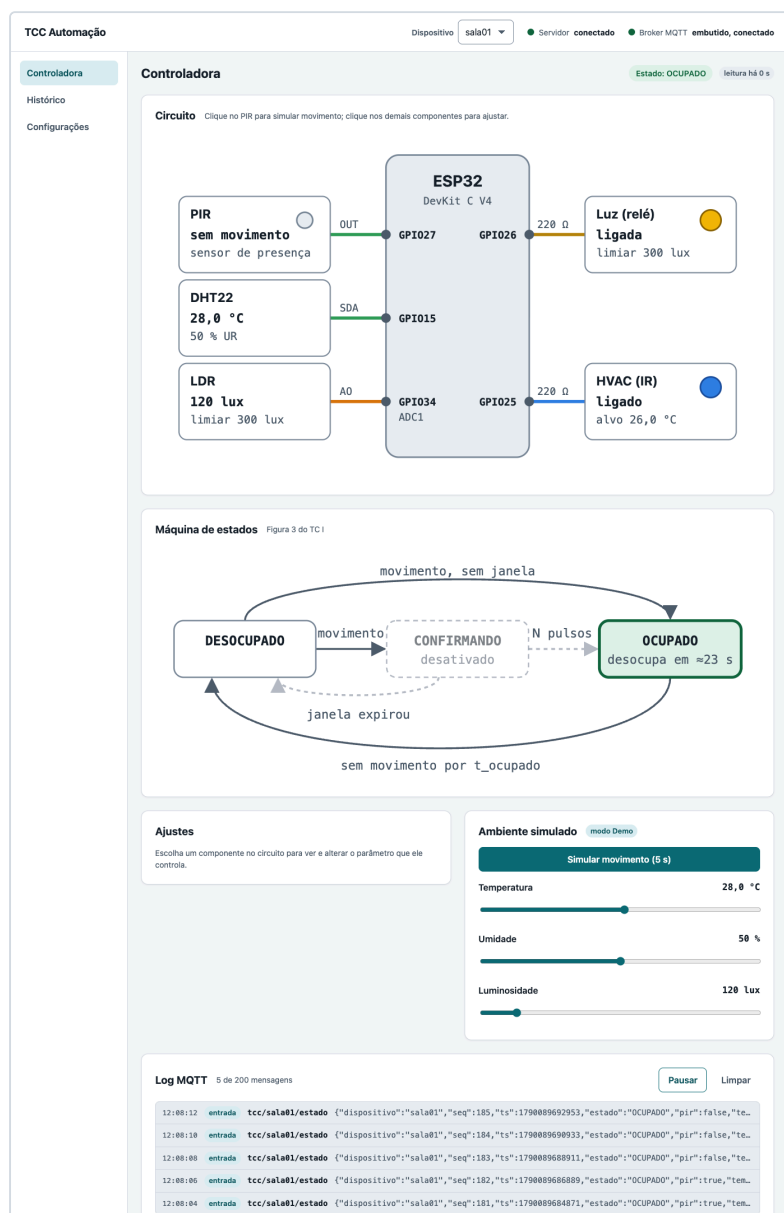
```
pnpm install
pnpm --filter @tcc/desktop exec install-electron # baixa o Electron
pnpm dev # dashboard em http://localhost:5173
pnpm desktop:dev # o mesmo, dentro da janela do aplicativo
pnpm desktop:dist # gera o instalador do sistema atual
```

## 5 Usando o aplicativo

Ao abrir, o aplicativo já sobe o broker, o banco e o dispositivo simulado `sala01` (modo Demo). No topo ficam o **seletor de dispositivo** e dois indicadores: a conexão com o servidor e a conexão com o broker MQTT. À esquerda, as três telas.

### 5.1 Controladora

A tela principal mostra o sistema em tempo real.



**Figura 3.** Controladora com a sala ocupada: luz (amarelo) e HVAC (azul) acesos, estado OCUPADO destacado na máquina de estados e as mensagens chegando no log MQTT.

- **Circuito:** o ESP32 com os sensores à esquerda (PIR, DHT22, LDR) e os atuadores à direita (luz e HVAC), com os pinos reais do firmware. Os valores lidos aparecem em cada componente e os LEDs acendem conforme o estado.

- **Máquina de estados:** a Figura 2 ao vivo, com o estado atual destacado e, depois que o movimento para, a contagem regressiva até desocupar.
- **Log MQTT:** as últimas 200 mensagens trocadas (entrada e saída), com botões para **pausar** e **limpar**.
- O selo no alto mostra o estado e há quanto tempo chegou a última leitura.

## 5.2 Ajustar parâmetros pelo diagrama

Clique num componente do circuito para abrir o painel de ajuste dele:

| Componente     | O que ajusta                                                     |
|----------------|------------------------------------------------------------------|
| PIR (presença) | Tempo até desocupar, janela de confirmação e detecções na janela |
| LDR e Luz      | Limiar de luminosidade                                           |
| DHT22 e HVAC   | Temperatura-alvo                                                 |

Ao clicar em **Aplicar**, o aplicativo publica a nova configuração no tópico `.../config` do dispositivo, que passa a usá-la na hora, sem reiniciar. Isso vale tanto para o modo Demo quanto para o firmware real.

## 5.3 Modo Demo

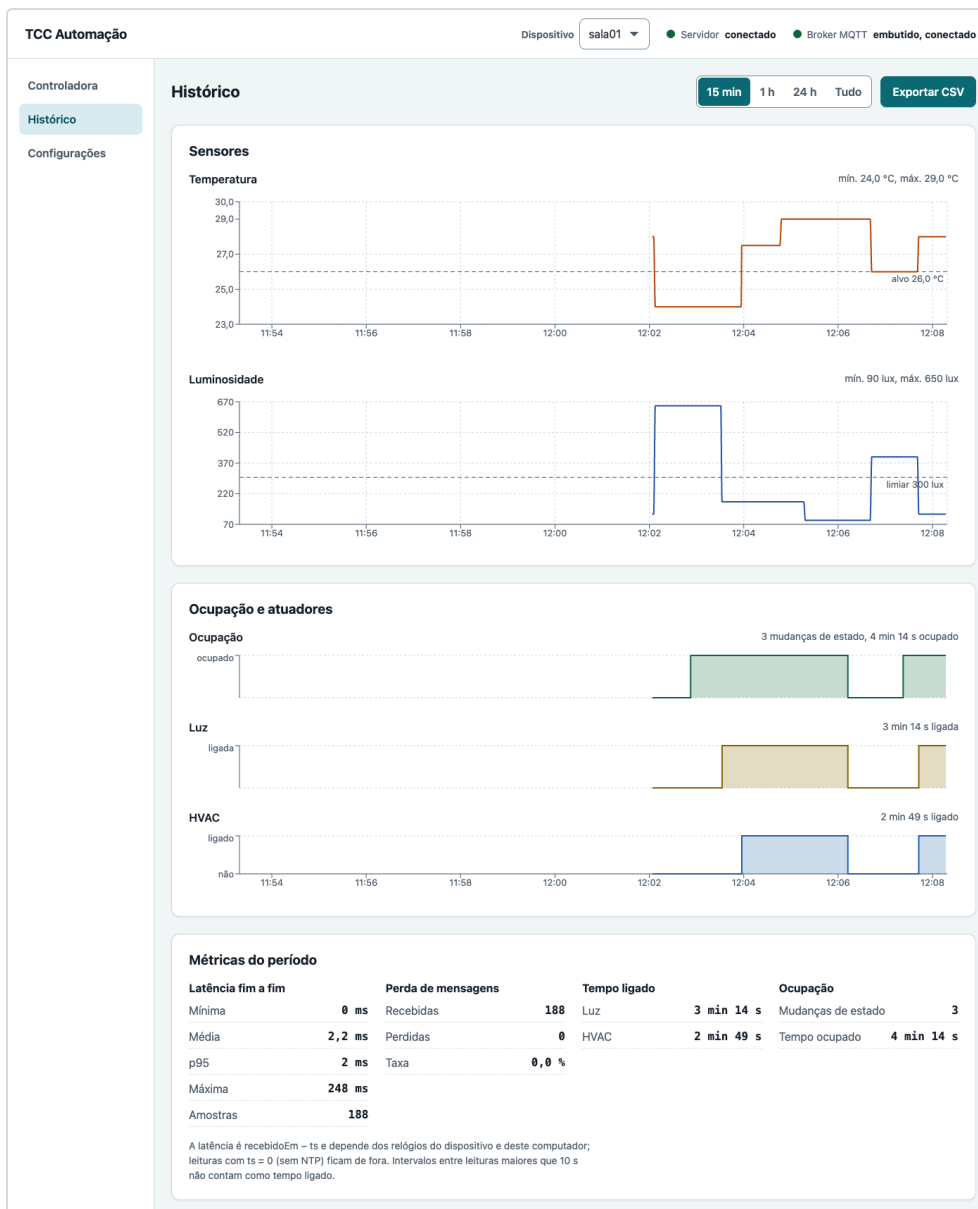
Com o Demo ligado, a Controladora mostra o cartão **Ambiente simulado**:

- **Simular movimento (5 s)** — o mesmo que clicar no PIR do diagrama: o sensor fica ativo por 5 segundos.
- **Deslizantes** de temperatura, umidade e luminosidade, para provocar as regras (por exemplo, subir a temperatura acima do alvo com a sala ocupada liga o HVAC).

### Roteiro rápido de demonstração

1. Clique no PIR: a sala passa a OCUPADO; a luz acende porque há 120 lux (abaixo de 300) e o HVAC liga porque faz 28 °C (acima de 26).
2. Leve a luminosidade para 600 lux: a luz apaga, porque já há claridade suficiente.
3. Clique no DHT22 e mude a temperatura-alvo para 30 °C: o HVAC desliga.
4. Espere 30 s sem movimento: a sala volta a DESOCUPADO e tudo desliga.

## 5.4 Histórico



**Figura 4.** Histórico: sensores com as linhas de temperatura-alvo e limiar de luz, faixas de ocupação e dos atuadores, e as métricas do período.

- Escolha o **período**: 15 min, 1 h, 24 h ou tudo. A tela se atualiza sozinha a cada 5 s.
- **Sensores**: temperatura e luminosidade no tempo, com a temperatura-alvo e o limiar de luz como linhas de referência.
- **Ocupação e atuadores**: quando a sala esteve ocupada e quando a luz e o HVAC estiveram ligados.
- **Métricas do período**: latência, perda de mensagens, tempo ligado e ocupação (definições na seção 7).
- **Exportar CSV** baixa as leituras do período para análise em planilha.

## 5.5 Configurações

TCC Automação

Dispositivo 

sala01

Servidor conectado

Broker MQTT embutido, conectado

Controladora

Histórico

Configurações

Configurações

Broker MQTT e modo Demo

Broker

☒ Embutido

O aplicativo sobe o próprio broker. Não precisa instalar nada.

☐ Externo

Recebe de um broker que já existe, como o Mosquitto ou um broker público.

☒ Modo Demo (dispositivo simulado)

Liga um dispositivo simulado (sala01) que publica leituras e responde à configuração.

Salvar e aplicar

Abrir em outro dispositivo

Na mesma rede local, abra um destes endereços no navegador do celular ou de outro computador:

http://192.168.1.3:3000

http://100.93.184.64:3000

http://192.168.139.3:3000

http://192.168.97.0:3000

http://192.168.147.0:3000

Portas em uso

|                        |                   |
|------------------------|-------------------|
| HTTP (dashboard e API) | 3000              |
| WebSocket              | 3000, caminho /ws |
| MQTT (broker embutido) | 62111             |

A porta padrão 1883 estava ocupada (por exemplo, pelo Mosquitto do Docker); o broker embutido usou a 62111.

**Figura 5.** Configurações: broker e modo Demo à esquerda; endereços para abrir em outro dispositivo e portas em uso à direita.

- **Broker embutido** (padrão): o aplicativo usa o próprio broker. É o modo do Demo.
- **Broker externo**: informe endereço, porta e raiz dos tópicos para receber de outro broker. O botão **Preencher com o broker público do TCC** coloca os dados usados pelo firmware no Wokwi.
- **Modo Demo**: liga ou desliga o dispositivo simulado (só funciona com o broker embutido).
- **Salvar e aplicar** troca tudo na hora, sem reiniciar o aplicativo.

## 5.6 Abrir no celular ou em outro computador

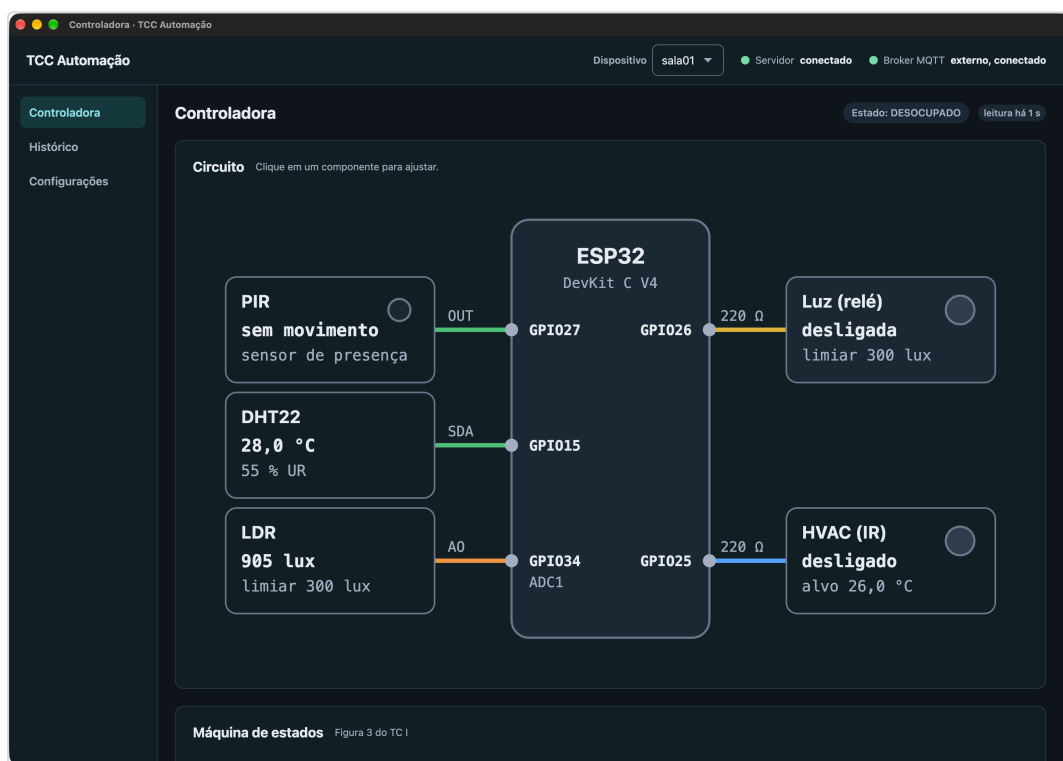
O cartão **Abrir em outro dispositivo** lista endereços como `http://192.168.1.3:3000`. Qualquer aparelho na mesma rede local abre o mesmo painel pelo navegador, com os dados em tempo real. Se a porta 3000 (HTTP) ou a 1883 (MQTT) estiver ocupada por outro programa, o aplicativo usa outra porta livre e mostra qual nesta tela.

## 6 Firmware real no Wokwi

O firmware do ESP32 roda no simulador Wokwi pela linha de comando ( `wokwi-cli` ). O ESP32 simulado só tem acesso à internet, por isso publica num **broker público** ( `broker.hivemq.com` ) com uma raiz de tópicos exclusiva do projeto ( `tcc-unip-7f3a9c` ).

### Passo a passo

- 1 Crie um token em `wokwi.com/dashboard/ci` e coloque no arquivo `.env` do repositório:  
`WOKWI_CLI_TOKEN=...`
- 2 Compile o firmware: `pnpm firmware:build`
- 3 No aplicativo, vá em **Configurações** → **Externo** → **Preencher com o broker público do TCC** → **Salvar e aplicar**.
- 4 Rode a simulação por 30 s: `pnpm firmware:sim`. A saída serial mostra `[mqtt]` conectado. (O plano gratuito do Wokwi CLI tem cerca de 50 min por mês, por isso cada execução é curta.)
- 5 A Controladora passa a mostrar os valores do firmware (dispositivo `sala01`).



**Figura 6.** Aplicativo em modo externo recebendo o firmware real do Wokwi (indicador “Broker MQTT externo, conectado”). O LDR mostra 905 lux, o valor do sensor simulado no Wokwi, diferente dos 120 lux do modo Demo.

### Também dá para usar o Wokwi pelo VS Code

Com as extensões PlatformIO e Wokwi, abra `firmware/diagram.json` e inicie a simulação: é possível clicar no PIR e mover os controles do DHT22 e do LDR com o mouse, e ver a reação no aplicativo.

## 7 Métricas e exportação

As métricas são calculadas pelo backend para qualquer período e aparecem na tela Histórico. Elas alimentam o Capítulo 4.

| Métrica            | Como é calculada                                                                                                                                                                                                                                                                                              |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Latência fim a fim | Hora de chegada no backend menos a hora em que o dispositivo gerou a leitura ( <code>recebidoEm</code> - <code>ts</code> , relógio do dispositivo via NTP). Mostra mínima, média, p95, máxima e o número de amostras. Leituras sem horário ( <code>ts</code> = 0 , NTP ainda não sincronizado) ficam de fora. |
| Perda de mensagens | Cada mensagem tem um número de sequência ( <code>seq</code> ). Uma lacuna (de 5 para 8, por exemplo) conta 2 mensagens perdidas. Mostra recebidas, perdidas e a taxa.                                                                                                                                         |
| Tempo ligado       | Soma dos intervalos em que a luz e o HVAC ficaram ligados. Intervalos maiores que 10 s entre leituras não contam (dispositivo fora do ar).                                                                                                                                                                    |
| Ocupação           | Número de mudanças de estado e tempo total ocupado.                                                                                                                                                                                                                                                           |

### Qual fonte usar no Capítulo 4

As métricas de desempenho (latência, perda) devem vir do **firmware real no Wokwi**, não do modo Demo: o simulador roda na mesma máquina, sem rede nem ruído de sensor. Note que a latência via Wokwi inclui a internet e a nuvem do simulador (em um teste, 2791 ms).

## Formato do CSV

O botão **Exportar CSV** gera um arquivo com uma linha por leitura. Booleanos vêm como 1/0; valores ausentes, vazios.

| Coluna                      | Conteúdo                                                            |
|-----------------------------|---------------------------------------------------------------------|
| recebidoEm                  | Hora de chegada no backend (milissegundos desde 1970)               |
| dispositivo , seq , ts      | Identificador, número de sequência e hora de geração no dispositivo |
| latenciaMs , perdas         | Latência da leitura e mensagens perdidas antes dela                 |
| estado , pir                | Estado de ocupação e se o PIR detectava movimento                   |
| temperatura , umidade , lux | Leituras dos sensores                                               |
| luz , hvac                  | Estado dos atuadores                                                |

```
recebidoEm,dispositivo,seq,ts,latenciaMs,perdas,estado,pir,temperatura,umidade,lux,luz,hvac  
1790041106121,sala01,0,1790041105834,287,0,DESOCUPADO,0,28,55,120,0,0
```

## 8 Contrato MQTT

---

Dispositivo e aplicativo conversam por dois tópicos. A `<raiz>` é `tcc` no broker embutido e `tcc-unip-7f3a9c` no broker público.

### Estado: dispositivo → aplicativo

Tópico `<raiz>/<dispositivo>/estado`, a cada 2 s e a cada mudança:

```
{ "dispositivo": "sala01", "seq": 42, "ts": 1790000000000, "estado": "OCUPADO",  
  "pir": true, "temperatura": 28.0, "umidade": 55.0, "lux": 120.5,  
  "luz": true, "hvac": true }
```

- `ts` : hora da leitura (NTP), ou 0 se o relógio ainda não sincronizou.
- `seq` : contador crescente, usado para detectar mensagens perdidas.
- `temperatura` e `umidade` podem vir `null` quando o DHT22 falha numa leitura.

### Configuração: aplicativo → dispositivo

Tópico `<raiz>/<dispositivo>/config`. Todos os campos são opcionais; o dispositivo aplica só os que vierem:

```
{ "tOcupadoMs": 10000, "janelaConfMs": 10000, "pulsosConf": 2,  
  "luxLimiar": 300, "tempAlvo": 26 }
```

Mensagens inválidas são descartadas pelo backend, com aviso no log, sem derrubar o sistema.

## 9 Decisões técnicas

Toda divergência em relação ao modelo do TC I está registrada em `docs/decisoes.md`, no formato contexto → decisão → consequência, para virar texto do TCC. Resumo:

| #           | Decisão                                       | Por quê                                                                                      |
|-------------|-----------------------------------------------|----------------------------------------------------------------------------------------------|
| D01         | ESP32 simulado no Wokwi                       | Sem hardware físico disponível; o código é o mesmo de uma placa real.                        |
| D02         | Máquina de estados isolada do hardware        | Permite testá-la no computador, sem placa.                                                   |
| D03         | Janela de confirmação opcional                | Desligada, reproduz a Figura 3; ligada, reduz falsos positivos.                              |
| D04–<br>D05 | Tarefas em dois núcleos, com mutex            | Comunicação não atrasa a decisão; a biblioteca MQTT não é segura entre tarefas.              |
| D06         | Restrições do ESP32/DHT22                     | LDR em pino ADC1 (o ADC2 para com Wi-Fi); DHT22 lido a cada 2 s.                             |
| D07         | Comparar durações, não instantes              | O contador de milissegundos estoura a cada ~49 dias; coberto por teste.                      |
| D08         | <code>ts</code> e <code>seq</code> no payload | Permitem medir latência e perda, comparáveis à literatura.                                   |
| D09         | Firmware no broker público                    | O Wokwi pela linha de comando só alcança a internet.                                         |
| D10         | Aplicativo desktop (Electron)                 | Abrir e demonstrar o sistema sem Docker nem instalação de serviços.                          |
| D11         | Broker MQTT embutido                          | Dispensa o Mosquitto; o externo continua disponível.                                         |
| D12         | SQLite no lugar do InfluxDB                   | Banco embutido no próprio programa; o CSV atende às análises.                                |
| D13         | Express + WebSocket dentro do app             | Preserva a arquitetura do TC I e permite abrir no navegador e no celular.                    |
| D14         | Modo Demo com a mesma máquina de estados      | Demonstração fiel ao firmware, validada pelos mesmos 9 testes.                               |
| D15         | Nome de arquivo do app sem acentos            | Com acentos, o app empacotado para macOS não abria; o nome exibido continua “TCC Automação”. |

## 10 Limitações

---

Pontos a declarar no texto do TCC:

- **Simulação, sem hardware físico:** os sensores do Wokwi não têm ruído real, e não há a rede local de um prédio.
- **Comando do ar-condicionado representado por um LED:** o Wokwi não simula infravermelho.
- **Luminosidade sem calibração:** a conversão do LDR para lux usa a fórmula do Wokwi.
- **Sem realimentação luz → sensor:** na simulação, acender a luz não aumenta a leitura do LDR; num ambiente real seria preciso histerese.
- **Latência via Wokwi inclui a internet** e a nuvem do simulador; não representa uma rede local.
- **Segurança MQTT fora do escopo:** brokers sem autenticação nem criptografia (seção 1.5 do TC I). No broker público, qualquer pessoa que conheça a raiz dos tópicos pode ler e publicar.
- **Instaladores sem assinatura digital;** o sistema operacional avisa na primeira abertura.
- **Um dispositivo por vez na tela:** o backend aceita vários, mas a interface mostra um seletor simples.

# 11 Para quem vai mexer no código

## Estrutura do repositório

```
firmware/          ESP32 + FreeRTOS (C++), simulado no Wokwi
  include/occupancy_fsm.h  máquina de estados, sem hardware
  src/main.cpp           as 4 tarefas
  test/fsm_test.cpp       testes da máquina de estados
apps/contrato/        formatos das mensagens e da API (zod)
apps/backend/         broker, banco, REST, WebSocket, modo Demo
apps/dashboard/       telas em React + Recharts
apps/desktop/         aplicativo Electron e instaladores
docs/                 contexto do TCC, decisões, evidências e este guia
```

## Comandos

| Comando                                                      | O que faz                                                    |
|--------------------------------------------------------------|--------------------------------------------------------------|
| <code>pnpm dev</code>                                        | Backend e dashboard em modo de desenvolvimento, no navegador |
| <code>pnpm desktop:dev</code>                                | O mesmo, dentro da janela do aplicativo                      |
| <code>pnpm desktop:dist</code>                               | Gera o instalador do sistema atual                           |
| <code>pnpm typecheck</code> · <code>pnpm test</code>         | Checagem de tipos e testes (158 testes)                      |
| <code>pnpm test:fsm</code>                                   | Os testes da máquina de estados em C++                       |
| <code>pnpm exp &lt;pasta&gt;</code>                          | Roda os roteiros dos experimentos do Capítulo 4              |
| <code>pnpm exp:latencia</code>                               | Mede latência e perda no pipeline real (CT-06)               |
| <code>pnpm firmware:build</code> · <code>firmware:sim</code> | Compila o firmware e roda 30 s no Wokwi                      |
| <code>pnpm mqtt:sub:publico</code>                           | Mostra as mensagens do broker público                        |

## Publicar uma nova versão

O GitHub Actions roda os testes a cada envio. Para gerar e publicar os instaladores, atualize a versão em `apps/desktop/package.json` e crie uma tag igual:

```
git tag v0.2.0 && git push origin v0.2.0
```

Em alguns minutos a página de Releases ganha o `.dmg` e o `.exe` da nova versão.